



Université Côte d'Azur
Master MIAGE
Projet Tuteuré

PICOXIA
Outil de Dictée Multilingue et
Enrichissement de Texte par IA

Rapport de Projet Tuteuré
Développement d'une application web de transcription vocale et
d'assistance à la rédaction

Informations du projet

Étudiants :	Abderrahmane Yousfi
	Yassine Hamaimou
	Yahia Ghany
	Amine Farikal
	Taha Abdelmohcine
Formation :	Master 1 MIAGE
Tuteur entreprise :	Benjamin TONI
Année universitaire :	2025 – 2026

Table des matières

Introduction Générale	3
1 Présentation du Projet	4
1.1 Contexte	4
1.2 Présentation de Picoxia	4
1.3 Problématique	4
1.4 Objectifs du projet	5
1.5 Périmètre du projet	5
2 Analyse des Besoins	6
2.1 Besoins fonctionnels	6
2.2 Besoins non fonctionnels	6
2.3 Cas d'utilisation	7
3 Étude Technique	8
3.1 Architecture générale	8
3.2 Frontend	8
3.3 Backend	9
3.4 Reconnaissance vocale	9
3.4.1 Solutions étudiées	9
3.4.2 Choix retenu : approche hybride à trois moteurs	10
3.5 IA et NLP pour l'enrichissement de texte	10
4 Gestion du Projet	11
4.1 Organisation de l'équipe	11
4.2 Méthode de travail	11
4.3 Planification	11
4.4 Répartition des tâches et travail collaboratif	11
4.5 Suivi de l'avancement	12
5 Conception	13
5.1 Architecture logicielle	13
5.2 Diagramme global	14
5.3 Flux de traitement	14
5.4 Conception de la base de données	15
5.5 Sécurité et gestion des données	16
6 Réalisation	17
6.1 Développement du module de dictée	17
6.2 Développement du module d'enrichissement	17

6.2.1	Choix du modèle Ollama	18
6.3	Frontend de l'enrichissement	18
6.4	Gestion du multilingue	19
6.5	Intégration MongoDB	19
7	Difficultés Rencontrées	21
8	Bilan et Perspectives	23
8.1	Bilan du projet	23
8.2	Compétences acquises	23
8.3	Limites actuelles	23
8.4	Améliorations futures	24
	Conclusion Générale	25
A	Captures d'écran	26
B	Guide d'installation	28
B.1	Prérequis	28
B.2	Installation du frontend	28
B.3	Installation du backend	28
B.4	Configuration des services externes	29
B.5	Verification	29
C	Guide utilisateur	30
C.1	Acces a l'application	30
C.2	Remplissage du formulaire	30
C.3	Utilisation de la dictée vocale	30
C.4	Enrichissement du texte	31
C.5	Sauvegarde du compte rendu	31
C.6	Resultat	31

Introduction Générale

L'évolution des technologies de traitement du langage naturel et de reconnaissance vocale a profondément transformé la manière dont les utilisateurs interagissent avec les systèmes informatiques. Aujourd'hui, la dictée vocale et les outils d'assistance à la rédaction permettent de simplifier la création de contenu tout en améliorant la productivité et l'accessibilité des applications numériques.

Dans ce contexte, l'entreprise Picoxia a proposé un projet visant à développer une solution capable de combiner reconnaissance vocale multilingue et enrichissement automatique de texte à l'aide de l'intelligence artificielle. L'objectif principal est de permettre à un utilisateur de dicter un message ou un compte rendu dans différentes langues, puis d'obtenir une version enrichie et mieux structurée du texte produit. Cette approche répond à des besoins croissants en matière de saisie rapide d'informations et d'assistance à la rédaction dans des environnements professionnels.

Le projet s'appuie sur une architecture moderne reposant sur React et TypeScript pour le développement de l'interface utilisateur, Flask pour les services backend et MongoDB pour la persistance des données. Afin de répondre aux exigences du cahier des charges, plusieurs technologies de reconnaissance vocale ont été étudiées et intégrées, notamment Whisper, Vosk et Web Speech API. L'enrichissement des textes est quant à lui assuré par des modèles d'intelligence artificielle exécutés localement via Ollama.

Au-delà de la réalisation technique, ce projet a représenté une opportunité d'appliquer des connaissances acquises au cours de la formation MIAGE dans un contexte concret, en abordant des problématiques liées à l'architecture logicielle, au développement full-stack, à l'intégration de solutions d'intelligence artificielle et à la gestion de données.

Ce rapport présente l'ensemble des travaux réalisés durant le projet. Il décrit d'abord le contexte et les besoins identifiés, puis les choix techniques effectués et l'architecture retenue. Les différentes étapes de conception et de développement sont ensuite détaillées avant de présenter les tests réalisés, les résultats obtenus ainsi que les principales difficultés rencontrées. Enfin, un bilan du projet et plusieurs perspectives d'amélioration sont proposés.

1 Présentation du Projet

1.1 Contexte

Ce projet a été réalisé dans le cadre d'un projet tutoré de Master MIAGE, en collaboration avec l'entreprise Picoxia, sous l'encadrement de Benjamin TONI. Le projet s'inscrit dans une logique de stage : développer une application web complète, en utilisant uniquement des technologies open source et gratuites, pour répondre à un besoin concret exprimé par l'entreprise.

1.2 Présentation de Picoxia

Picoxia est un logiciel d'acquisition radiographique et de pré-diagnostic assisté par intelligence artificielle, destiné aux cabinets et cliniques vétérinaires. En production, il permet aux vétérinaires de réaliser et d'analyser des examens radiologiques, avec une assistance automatique à l'interprétation des images.

Notre travail s'inscrit dans cette démarche en proposant deux fonctionnalités complémentaires : un formulaire de contact multilingue avec dictée vocale, et un module d'enrichissement automatique de rapports radiographiques. L'objectif est de simplifier et d'accélérer la rédaction de comptes rendus au sein du logiciel, en permettant aux utilisateurs de dicter leurs observations dans leur langue et d'obtenir automatiquement une version structurée et professionnelle du texte produit.

Langues supportées

Français, anglais, allemand, serbe, grec, néerlandais, russe, ukrainien, italien, finnois, portugais, slovaque, espagnol et tchèque.

1.3 Problématique

Dans un cabinet vétérinaire, les comptes rendus d'examens (radiologie, consultations, etc.) sont souvent rédigés rapidement, parfois en dictant à voix haute, ce qui donne des textes au style oral, avec des fautes ou des abréviations. Ces comptes rendus doivent ensuite être mis au propre pour être lisibles et professionnels. De plus, le personnel ou les clients peuvent parler des langues différentes, ce qui complique la communication si l'outil ne fonctionne qu'en français.

La problématique du projet peut donc se résumer ainsi : comment permettre à un utilisateur de dicter un texte dans sa langue, puis d'obtenir automatiquement une version structurée et professionnelle de ce texte, tout en respectant la confidentialité des données médicales et sans dépendre de services payants ?

1.4 Objectifs du projet

Les objectifs principaux du projet sont les suivants :

- permettre la dictée vocale dans 14 langues, avec plusieurs moteurs de reconnaissance vocale selon le contexte (en ligne, hors ligne, avec ou sans backend) ;
- enrichir automatiquement un texte dicté grâce à un modèle d'intelligence artificielle, en le corrigeant et en le structurant ;
- sauvegarder les comptes rendus (texte original et texte enrichi) dans une base de données ;
- utiliser uniquement des technologies open source et, dans la mesure du possible, fonctionnant en local pour préserver la confidentialité des données.

1.5 Périmètre du projet

Plutôt que de séparer le projet en deux applications distinctes, Picoxia a été conçu comme un formulaire de contact unique et unifié, qui regroupe l'ensemble des fonctionnalités. L'utilisateur y choisit un type de demande (bug logiciel, bug matériel, assistance, autre) et une catégorie (médical, administratif, technique, facturation, général), sélectionne sa langue, saisit un sujet (avec autocomplétion sur des suggestions courantes, par exemple des races d'animaux ou des symptômes), puis dicte ou écrit le contenu de sa demande dans un champ message. Ce champ peut être un message de contact classique ou un compte rendu vétérinaire (un exemple de compte rendu type est proposé comme texte indicatif dans le champ).

Une fois le message rédigé, l'utilisateur peut, de façon facultative, cliquer sur « Enrichir avec l'IA » pour obtenir une version corrigée et structurée du texte, comparer les deux versions, puis choisir de garder l'original ou la version enrichie avant d'envoyer le formulaire.

Exemple de compte rendu

Un exemple de compte rendu type est utilisé comme texte indicatif dans le champ message. Il décrit un examen radiologique vétérinaire (thorax, abdomen, bassin) avec des mesures précises (par exemple l'angle de Norberg-Olsson pour les hanches) et des sections « Conclusions » et « Recommandations » à compléter. Ce type de document illustre bien l'objectif du module d'enrichissement : préserver les données médicales chiffrées tout en structurant et complétant le texte autour.

Le périmètre du projet ne couvre pas le déploiement en production multi-utilisateurs : l'application a été conçue et testée comme un prototype fonctionnel, destiné à démontrer la faisabilité de la solution.

2 Analyse des Besoins

2.1 Besoins fonctionnels

À partir du formulaire unifié décrit au chapitre précédent, les besoins fonctionnels suivants ont été identifiés :

- l'utilisateur doit pouvoir choisir une langue parmi les 14 langues supportées ;
- l'utilisateur doit pouvoir choisir un type de demande (bug logiciel, bug matériel, assistance, autre) et une catégorie (médical, administratif, technique, facturation, général) ;
- l'utilisateur doit pouvoir saisir un sujet, avec des suggestions d'autocomplétion sur des éléments courants (races d'animaux, symptômes) ;
- l'utilisateur doit pouvoir dicter un texte au micro, et voir ce texte apparaître dans le champ message du formulaire ;
- l'utilisateur doit pouvoir choisir le mode de reconnaissance vocale (Auto, Haute précision, Offline, Web Speech) selon ses contraintes (connexion, navigateur, confidentialité) ;
- l'utilisateur doit pouvoir démarrer et arrêter la dictée à tout moment ;
- l'utilisateur doit pouvoir, de façon facultative, enrichir le texte du message grâce à l'IA, en un clic ;
- l'utilisateur doit pouvoir comparer le texte original et le texte enrichi, et choisir d'utiliser la version enrichie ou de la rejeter ;
- l'utilisateur doit pouvoir envoyer le formulaire, ce qui sauvegarde le texte original et le texte enrichi (identiques si l'enrichissement n'a pas été utilisé), la langue et le type de demande dans une base de données.

2.2 Besoins non fonctionnels

En plus des fonctionnalités, plusieurs contraintes non fonctionnelles ont guidé les choix techniques présentés au chapitre 3 :

- **Confidentialité** : les données traitées (comptes rendus vétérinaires) peuvent concerner des patients et leurs propriétaires. La solution doit donc privilégier des traitements locaux (transcription, enrichissement) plutôt que des services cloud.
- **Coût** : le projet doit utiliser uniquement des outils open source et gratuits (Vosk, Whisper, Ollama, MongoDB, React, Flask), sans abonnement à une API payante.
- **Disponibilité partielle** : l'application doit rester utilisable même si certains services locaux (backend Flask, Ollama) ne sont pas lancés, en informant clairement l'utilisateur plutôt qu'en échouant silencieusement.

- **Multilingue** : l'interface et la reconnaissance vocale doivent fonctionner de manière cohérente dans les 14 langues, sans traduction manuelle répétitive à chaque évolution de l'interface.
- **Fiabilité du contenu enrichi** : le module d'IA ne doit pas inventer ou modifier les données médicales originales (dosages, mesures, durées), il doit seulement corriger et structurer le texte autour.

2.3 Cas d'utilisation

Le formulaire étant unique, les deux scénarios suivants correspondent en réalité au même parcours, avec ou sans l'étape d'enrichissement.

Cas 1 : envoyer une demande simple. L'utilisateur sélectionne sa langue, choisit un type de demande (par exemple « bug logiciel ») et une catégorie, saisit un sujet, dicte ou écrit son message, puis envoie directement le formulaire sans utiliser l'enrichissement.

Cas 2 : rédiger et enrichir un compte rendu. L'utilisateur choisit le type de demande « assistance » et la catégorie « médical », dicte un compte rendu (par exemple un compte rendu d'examen radiologique vétérinaire), clique sur « Enrichir avec l'IA », compare le texte original et le texte enrichi dans le panneau affiché, choisit « Utiliser » ou « Ignorer », puis envoie le formulaire. Dans les deux cas, le texte original et le texte enrichi (identiques si l'enrichissement n'a pas été utilisé) sont sauvegardés.

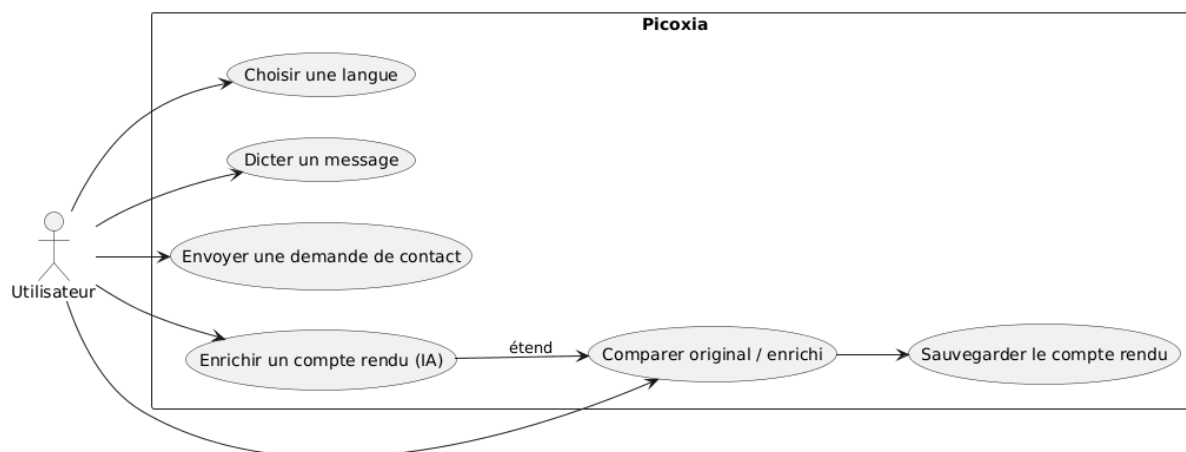


FIGURE 2.1 – Diagramme de cas d'utilisation de Picoxia

3 Étude Technique

Ce chapitre présente les technologies retenues pour développer Picoxia et explique les raisons de ces choix. L'objectif n'est pas de décrire en détail chaque outil, mais de justifier pourquoi ils correspondent aux besoins du projet : une application web multilingue, capable de fonctionner avec des outils open source et gratuits, et offrant une dictée vocale fiable dans 14 langues.

3.1 Architecture générale

Picoxia repose sur une architecture classique en trois couches. Le frontend, développé en React et TypeScript, gère l'interface utilisateur, la sélection de la langue et la dictée vocale. Le backend, développé en Python avec Flask, expose une API qui prend en charge la transcription audio via Whisper et le traitement du texte. La base de données MongoDB stocke les messages, les comptes rendus et les résultats enrichis.

Une particularité de Picoxia est que la reconnaissance vocale peut se faire de deux façons différentes selon le contexte : soit côté serveur avec Whisper (haute précision, mais nécessite que le backend Flask soit lancé), soit directement dans le navigateur avec Vosk (mode hors ligne, sans dépendance au serveur). Cette approche hybride est détaillée dans la section 3.4.

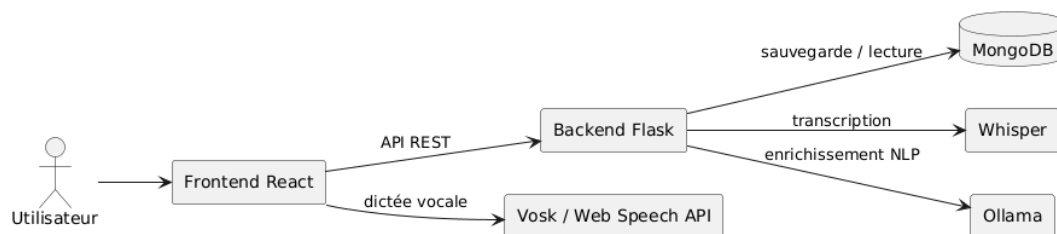


FIGURE 3.1 – Architecture générale de Picoxia (Frontend, Backend, moteurs de reconnaissance vocale, base de données)

3.2 Frontend

Le frontend est développé avec React 19 et TypeScript. Ce choix a été fait pour plusieurs raisons. React permet de construire une interface réactive, organisée en composants réutilisables, ce qui est utile pour gérer les 14 langues et les différents modes de dictée. TypeScript apporte un typage statique qui réduit les erreurs lors du développement, en particulier sur la gestion des clés de traduction et des configurations de langue.

Pour les composants d'interface, le projet utilise la bibliothèque RSuite, qui propose des composants prêts à l'emploi (sélecteurs, boutons, formulaires) et permet de gagner du temps sur la mise en forme tout en gardant une interface cohérente. Le frontend s'appuie

aussi sur Vite comme outil de build et de serveur de développement, pour des temps de rechargement rapides.

Internationalisation

Les 14 langues sont gérées via un système de traduction centralisé : un fichier source en français (`fr.ts`) sert de référence, et un script (`npm run translate`) traduit automatiquement les clés manquantes vers les 13 autres langues via l'API gratuite MyMemory. Cela évite de devoir traduire manuellement chaque texte de l'interface.

3.3 Backend

Le backend est développé en Python avec le micro-framework Flask. Ce choix a été motivé par sa simplicité : Flask permet de mettre en place rapidement une API REST sans surcouche inutile, ce qui convenait bien à un projet où le backend sert principalement à exposer un endpoint de transcription (`/api/transcribe`) et un endpoint de vérification de disponibilité (`/api/health`).

Pour le stockage des données (messages du formulaire de contact, comptes rendus enrichis), le projet utilise MongoDB. Cette base NoSQL a été choisie car les documents traités (messages, comptes rendus, résultats d'enrichissement) ont une structure qui peut varier légèrement d'une langue ou d'un cas à l'autre, ce qui correspond bien au modèle de documents JSON de MongoDB.

3.4 Reconnaissance vocale

Le choix du moteur de reconnaissance vocale (Speech-to-Text) est l'un des choix techniques les plus importants du projet, car il a un impact direct sur la précision de la dictée, la rapidité de la transcription, le fonctionnement hors ligne et la confidentialité des données. Trois solutions ont été étudiées : Whisper, Vosk et la Web Speech API.

● 3.4.1 Solutions étudiées

Whisper : Whisper est un modèle de transcription développé par OpenAI, reconnu pour sa très bonne précision et son support natif d'un grand nombre de langues (99 langues). Dans Picoxia, il est utilisé côté serveur : l'audio enregistré est envoyé au backend Flask via l'endpoint `/api/transcribe`, qui retourne le texte transcrit une fois l'enregistrement terminé. Son principal inconvénient est qu'il nécessite de lancer un backend Python local et qu'il ne fournit pas de résultats intermédiaires en temps réel.

Vosk : Vosk est un moteur de reconnaissance vocale open source, léger et capable de fonctionner entièrement hors ligne. Dans Picoxia, il est intégré directement dans le navigateur via `vosk-browser`, qui exécute la reconnaissance en WebAssembly. Les modèles de langue (40 à 80 Mo selon la langue) sont téléchargés automatiquement au premier usage et mis en cache. Sa précision est correcte mais reste inférieure à celle de Whisper, en particulier sur les langues moins courantes parmi les 14 supportées.

Web Speech API : c'est une interface native des navigateurs, standardisée par le W3C et implémentée dans Chrome et Edge. Concrètement, le navigateur envoie l'audio capturé aux serveurs de Google, qui renvoient le texte transcrit avec des résultats intermédiaires en temps réel. C'est la solution la plus simple à intégrer (aucune installation,

aucun modèle à télécharger), mais elle nécessite une connexion internet, ne fonctionne que sur Chrome et Edge, et envoie l'audio à un service tiers, ce qui pose une question de confidentialité pour des données médicales.

TABLE 3.1 – Comparaison des trois moteurs de reconnaissance vocale étudiés

Critère	Whisper	Vosk	Web Speech API
Localisation	Serveur Flask local	Navigateur (WASM)	Serveurs Google (cloud)
Connexion requise	Réseau local	Non	Oui (internet)
Précision	Très haute	Moyenne	Haute
Langues supportées	99 langues	Modèle par langue	~50 langues
Latence	Après enregistrement	Temps réel	Temps réel
Résultats intermédiaires	Non	Oui	Oui
Dépendance externe	Backend Python requis	Modèle ~50 Mo	Aucune installation
Confidentialité	Audio local	Audio local	Audio envoyé à Google
Support navigateur	Tous	Tous	Chrome / Edge uniquement

● 3.4.2 Choix retenu : approche hybride à trois moteurs

Plutôt que de choisir un seul moteur, Picoxia combine les trois solutions pour profiter des avantages de chacune. L'utilisateur peut choisir entre plusieurs modes dans l'interface :

- **Auto** : le système essaie d'utiliser Whisper si le backend est joignable. En cas d'indisponibilité, il bascule automatiquement vers Vosk, avec un message expliquant le changement à l'utilisateur.
- **Haute précision** : seul Whisper est utilisé. Si le backend ne répond pas, une erreur est affichée plutôt que de basculer silencieusement.
- **Offline** : seul Vosk est utilisé, ce qui permet une dictée 100 % locale, sans dépendre du serveur de transcription.
- **Web Speech** : utilise la reconnaissance vocale du navigateur (Chrome/Edge), avec affichage du texte en temps réel au fur et à mesure de la dictée.

Pourquoi garder les trois moteurs ?

Chaque moteur répond à un besoin différent : Whisper pour la précision lorsque le backend est disponible, Vosk pour un fonctionnement garanti hors ligne et respectueux de la confidentialité, et Web Speech API pour un retour visuel immédiat sans aucune installation. Plutôt que d'imposer un compromis unique, Picoxia laisse le choix à l'utilisateur selon son contexte d'utilisation.

3.5 IA et NLP pour l'enrichissement de texte

Pour la partie enrichissement (correction, reformulation, structuration des comptes rendus), l'objectif est de transformer un texte brut dicté (style oral, fautes, abréviations) en un rapport structuré et professionnel, tout en restant 100 % local pour des raisons de confidentialité des données médicales.

La solution retenue repose sur **Ollama**, un outil open source qui permet de faire tourner des grands modèles de langage (LLM) localement, sans connexion internet ni coût d'API. Le modèle utilisé est **qwen2.5:3b**. Ce choix et les alternatives étudiées sont détaillés dans le chapitre consacré à la réalisation.

4 Gestion du Projet

4.1 Organisation de l'équipe

Le projet a été réalisé par un groupe d'étudiants, chacun en charge d'une ou plusieurs parties de l'application (frontend, dictée vocale, enrichissement IA, intégration MongoDB). La répartition des tâches s'est faite de manière organique au fil des semaines, en fonction des besoins du moment et des compétences de chacun, plutôt que par une attribution fixe et définitive dès le début du projet.

4.2 Méthode de travail

Le projet a suivi un rythme itératif simple : une réunion hebdomadaire avec le tuteur permettait de présenter l'incrément réalisé pendant la semaine, et le tuteur donnait en retour les objectifs ou priorités pour la semaine suivante. Cette organisation se rapproche d'un fonctionnement de type Scrum simplifié, sans toutefois suivre formellement tous les rituels (pas de sprint planning ou de rétrospective formalisés), mais avec le même principe de cycles courts et de retours réguliers.

Cycle hebdomadaire

Chaque semaine suivait le même schéma : développement des fonctionnalités définies la semaine précédente, réunion avec le tuteur pour faire le point sur l'avancement, puis définition des objectifs de la semaine suivante. Ce rythme a permis de garder une vision claire de la progression du projet et d'ajuster rapidement les priorités.

4.3 Planification

La planification du projet s'est faite de façon glissante, semaine par semaine, plutôt que par un planning détaillé fixé en début de projet. Les premières semaines ont été consacrées à la mise en place de la base de l'application (formulaire de contact, première intégration de la dictée vocale), avant d'enchaîner sur les fonctionnalités plus avancées : enrichissement par IA, intégration MongoDB, ajout de moteurs de reconnaissance vocale supplémentaires (Vosk, puis Web Speech API), et améliorations de l'interface.

4.4 Répartition des tâches et travail collaboratif

Le travail a été organisé avec Git, en utilisant une branche dédiée par fonctionnalité ou par sujet de travail. Chaque membre du groupe développait sa partie sur sa propre branche, avant de l'intégrer au reste du projet. Cette organisation par branches a permis à plusieurs personnes de travailler en parallèle sur des sujets différents : par exemple,

l'intégration de Web Speech API, l'amélioration du module NLP avec Ollama, l'intégration de MongoDB, ou des améliorations d'interface (UI/UX), ont pu avancer indépendamment avant d'être fusionnées.

Organisation par branches

Concrètement, le projet a vu cohabiter plusieurs branches actives en parallèle, chacune correspondant à un sujet précis (par exemple une branche pour les améliorations UI/UX, une pour l'intégration du backend NLP avec Ollama, une pour l'intégration MongoDB, une pour l'ajout de la Web Speech API). Cette granularité a permis de limiter les conflits lors des fusions et de garder un historique clair de qui a travaillé sur quoi.

4.5 Suivi de l'avancement

Le suivi de l'avancement s'est fait principalement via les réunions hebdomadaires avec le tuteur et via l'historique des branches et des commits sur le dépôt Git. Cette approche, bien que moins formelle qu'un outil de gestion de projet dédié (type Trello ou Jira), a été suffisante pour un projet de cette taille : elle a permis de visualiser facilement qui travaillait sur quelle partie, et d'identifier rapidement les fonctionnalités terminées ou en cours.

5 Conception

Ce chapitre détaille la conception de Picoxia : l'architecture logicielle, le flux de traitement complet d'une dictée, et la structure de la base de données utilisée pour stocker les comptes rendus.

5.1 Architecture logicielle

L'application est découpée en trois grandes parties qui communiquent entre elles : le frontend React, le backend Flask, et les moteurs de transcription/enrichissement. Le frontend est responsable de l'interface, de la capture audio et du choix du moteur de reconnaissance vocale. Le backend expose une API REST qui regroupe trois familles d'endpoints :

- la transcription audio (`/api/transcribe`, `/api/health`);
- l'enrichissement NLP via Ollama (`/api/enrich`, `/api/enrich/health`);
- la persistance des comptes rendus en MongoDB (`/api/reports`).

Cette séparation en blueprints Flask indépendants permet à chaque module (transcription, enrichissement, sauvegarde) d'évoluer séparément, et facilite les tests : chaque endpoint peut être testé isolément avec curl ou Postman avant d'être branché au frontend.

5.2 Diagramme global

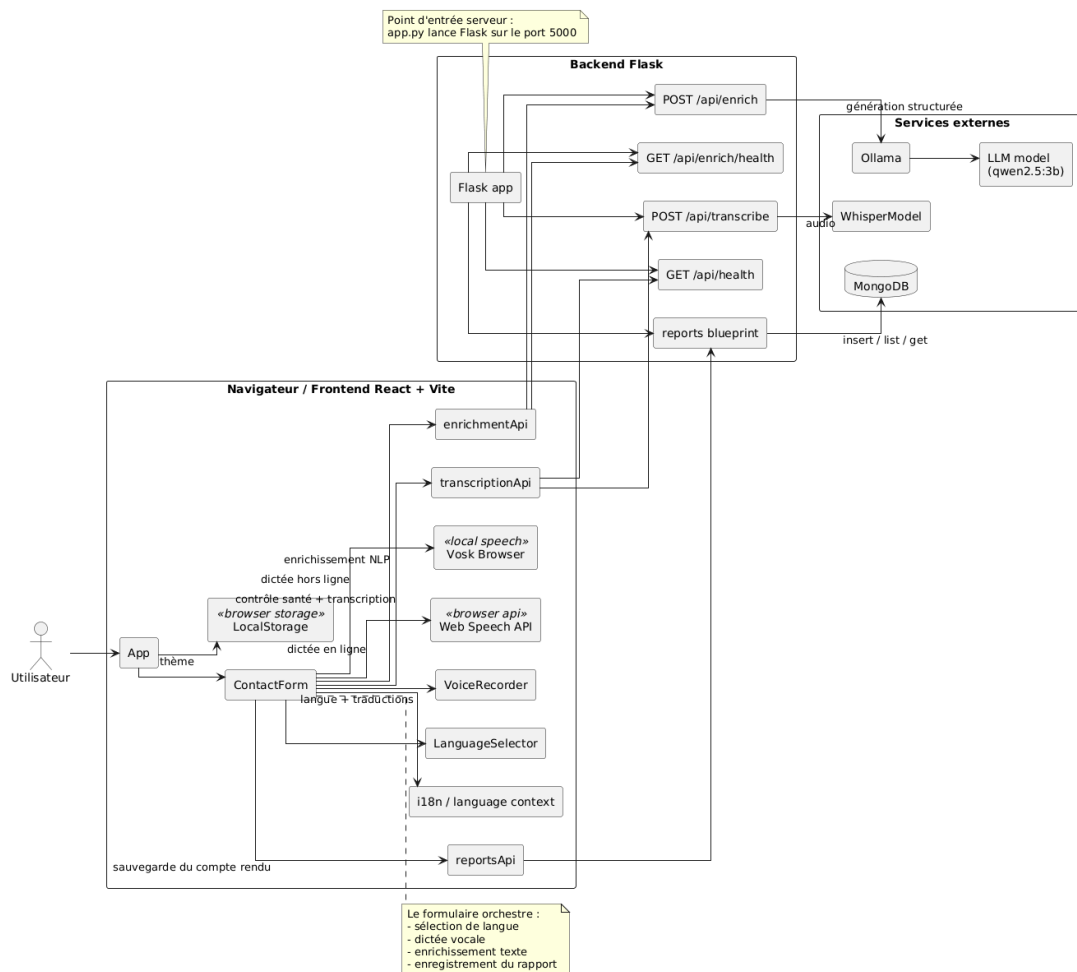


FIGURE 5.1 – Vue d'ensemble de l'architecture Picoxia : frontend, moteurs de reconnaissance vocale, backend Flask, Ollama et MongoDB

5.3 Flux de traitement

Le scénario principal de Picoxia suit les étapes suivantes :

1. L'utilisateur choisit une langue et un mode de reconnaissance vocale (Auto, Haute précision, Offline ou Web Speech).
2. Il dicte un texte, qui est transcrit soit localement (Vosk, Web Speech), soit via le backend (Whisper).
3. Le texte transcrit apparaît dans le champ du formulaire et peut être complété manuellement ou par une nouvelle dictée.
4. L'utilisateur peut cliquer sur « Enrichir avec l'IA » : le texte est envoyé au backend, qui l'envoie à Ollama via le prompt d'enrichissement.
5. Le backend renvoie le texte enrichi. Un panneau de comparaison affiche le texte original et le texte enrichi côte à côte.
6. L'utilisateur choisit « Utiliser » (le texte enrichi remplace le message) ou « Ignorer ».

7. Le compte rendu final (texte original et/ou enrichi, langue, type de demande) est sauvegardé via `POST /api/reports` dans MongoDB.

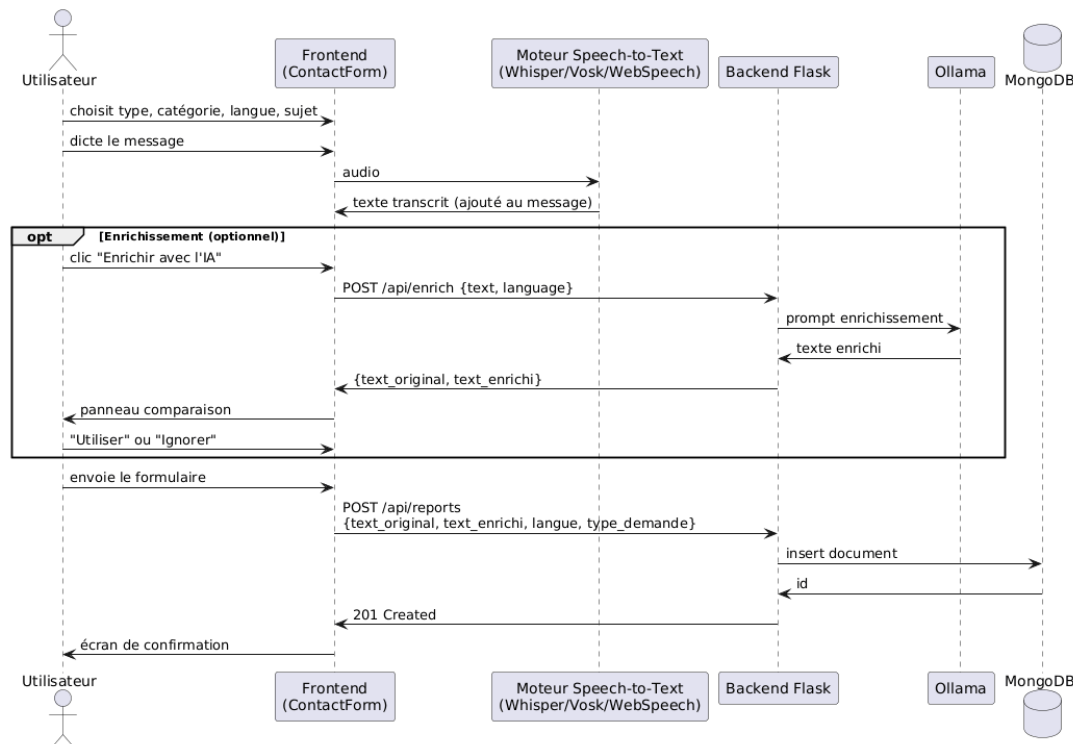


FIGURE 5.2 – Diagramme de séquence : du choix de la langue à la sauvegarde du compte rendu enrichi

5.4 Conception de la base de données

La base de données MongoDB (base `projet_miage`, collection `reports` par défaut) stocke les comptes rendus sous forme de documents JSON. Le choix d'une base NoSQL orientée documents permet de stocker dans une même entrée à la fois le texte original, le texte enrichi et leurs métadonnées, sans avoir besoin de plusieurs tables liées comme dans une base relationnelle.

TABLE 5.1 – Structure d'un document de la collection `reports`

Champ	Description
<code>text_original</code>	Texte dicté par l'utilisateur, avant enrichissement
<code>text_enrichi</code>	Texte produit par le module NLP (Ollama)
<code>langue</code>	Code de la langue du compte rendu (parmi les 14 langues)
<code>type_demande</code>	Catégorie de la demande (ex. <code>compte_rendu</code>)
<code>date</code>	Date de création, au format ISO 8601

La configuration de la connexion (URI MongoDB, nom de la base, nom de la collection) est entièrement paramétrable via des variables d'environnement (`MONGODB_URI`, `MONGODB_DB`, `MONGODB_COLLECTION`), ce qui permet de basculer facilement entre une base locale (développement) et une base distante (déploiement) sans modifier le code.

5.5 Sécurité et gestion des données

Le projet traite des données potentiellement sensibles, puisqu'il s'agit de comptes rendus vétérinaires pouvant contenir des informations sur des patients (animaux et propriétaires). Plusieurs choix de conception ont été faits en conséquence :

- la transcription Whisper et l'enrichissement Ollama fonctionnent tous les deux en local, donc l'audio et le texte ne sont jamais envoyés à un service tiers ;
- le mode Web Speech, qui envoie l'audio à Google, est proposé comme option facultative et non comme mode par défaut, justement pour cette raison ;
- si MongoDB n'est pas joignable, les endpoints renvoient une erreur explicite plutôt que de perdre silencieusement les données.

Confidentialité

L'approche 100 % locale (Vosk pour la dictée hors ligne, Whisper et Ollama pour la transcription et l'enrichissement) est un choix de conception central du projet : elle évite l'envoi de données médicales vers des services cloud externes, ce qui est un point important dans un contexte de conformité RGPD.

6 Réalisation

Ce chapitre décrit concrètement ce qui a été développé pour chaque module de l'application : la dictée vocale, l'enrichissement par IA, la gestion du multilingue et la sauvegarde en base de données.

6.1 Développement du module de dictée

Le module de dictée repose sur le composant `VoiceRecorder.tsx` et sur plusieurs hooks React dédiés à chaque moteur de reconnaissance vocale : `useVoskRecognition.ts`, `useWhisperRecognition.ts` et `useWebSpeechRecognition.ts`. Chaque hook expose la même interface (`start`, `stop`, `status`), ce qui permet au composant principal de basculer d'un moteur à l'autre sans changer sa logique d'affichage.

Pipeline Vosk. La capture audio se fait via `getUserMedia` (flux mono, avec annulation d'écho et réduction de bruit). Un `AudioContext` connecte ce flux à deux branches : un `ScriptProcessorNode` qui transmet les buffers audio à `KaldiRecognizer` pour la reconnaissance, et un `AnalyserNode` qui alimente une visualisation de la forme d'onde en temps réel. Le recognizer émet deux types d'événements : `result` pour le texte final, et `partialresult` pour le texte intermédiaire affiché pendant la dictée.

Pipeline Whisper. L'enregistrement audio est envoyé en POST multipart vers `/api/transcribe`. Le backend traite l'audio avec Whisper et renvoie le texte transcrit, qui est ajouté au champ du formulaire une fois la requête terminée.

Pipeline Web Speech. Le hook `useWebSpeechRecognition.ts` détecte le support du navigateur (`SpeechRecognition` ou `webkitSpeechRecognition`), puis configure la reconnaissance avec `continuous = true` et `interimResults = true`. Les résultats intermédiaires (`isFinal = false`) sont affichés en gris pendant que l'utilisateur parle, et le résultat final (`isFinal = true`) est ajouté au message.

Un quatrième mode « Web Speech » a été ajouté dans le sélecteur de mode du composant `VoiceRecorder.tsx`, et le verrou de dictée (`isDictationLock`) de `ContactForm.tsx` a été étendu pour bloquer le formulaire pendant une session Web Speech, de la même façon que pour Whisper et Vosk.

6.2 Développement du module d'enrichissement

Le module d'enrichissement repose sur Ollama, exécuté localement avec le modèle `qwen2.5:3b` (environ 1,9 Go). Côté backend, deux endpoints ont été ajoutés dans `backend/app.py` :

- GET `/api/enrich/health` : vérifie qu'Ollama est lancé et que le modèle demandé est disponible ;
- POST `/api/enrich` : reçoit `{ text, language }`, envoie le texte au modèle via un prompt spécialisé, et retourne `{ text_enrichi, text_original, langue, model }`.

Le prompt d'enrichissement demande au modèle de corriger les fautes d'orthographe et de grammaire, d'enrichir le vocabulaire avec des termes vétérinaires techniques, de structurer le texte en sections (Anamnèse, Examen, Diagnostic, Traitement), de préserver toutes les données médicales originales (dosages, mesures, durées), et de répondre dans la même langue que le texte d'entrée. Le modèle et l'hôte Ollama sont configurables via les variables d'environnement `OLLAMA_MODEL` et `OLLAMA_HOST`.

● 6.2.1 Choix du modèle Ollama

Plusieurs modèles ont été testés avant de retenir `qwen2.5:3b`.

TABLE 6.1 – Modèles Ollama testés pour l'enrichissement

Modèle	Taille	Avantages	Inconvénients
<code>qwen2.5:7b</code>	~4,7 Go	Très bon en français et excellent suivi d'instructions.	Lent sans GPU.
<code>llama3.2:3b</code>	~2 Go	Très léger et rapide sur CPU.	Moins bon en français et en raisonnement.
<code>qwen2.5:3b</code> (retenu)	~1.9 Go	Bon français et bon suivi d'instructions pour sa taille.	robuste avec précision acceptable.

Lors des tests, `qwen2.5:3b` a été retenu car il suit fidèlement le prompt (routage correct des sections, omission des sections vides, pas d'hallucination) tout en offrant une latence nettement plus faible, compatible avec un usage en temps réel. À titre de comparaison, `llama3.2:3b` inventait des symptômes absents du texte original (par exemple une toux non mentionnée) et mélangeait les sections du rapport, ce qui était inacceptable pour un document médical, tandis que `qwen2.5:7b`, bien que très fiable, présentait une latence trop élevée pour être utilisé localement.

6.3 Frontend de l'enrichissement

Côté frontend, un nouveau service `enrichmentApi.ts` encapsule l'appel à `POST /api/enrich`, avec un timeout de 60 secondes pour laisser le temps au modèle de répondre. Dans `ContactForm.tsx`, une section « Enrichissement IA » a été ajoutée avec :

- un bouton « Enrichir avec l'IA », désactivé si le message est vide ou si une dictée est en cours ;
- un indicateur de chargement pendant le traitement ;
- un panneau de comparaison avant/après, avec le texte original à gauche et le texte enrichi à droite, et deux boutons « Utiliser » (remplace le message) et « Ignorer » (ferme le panneau) ;
- un message d'erreur si Ollama n'est pas lancé.

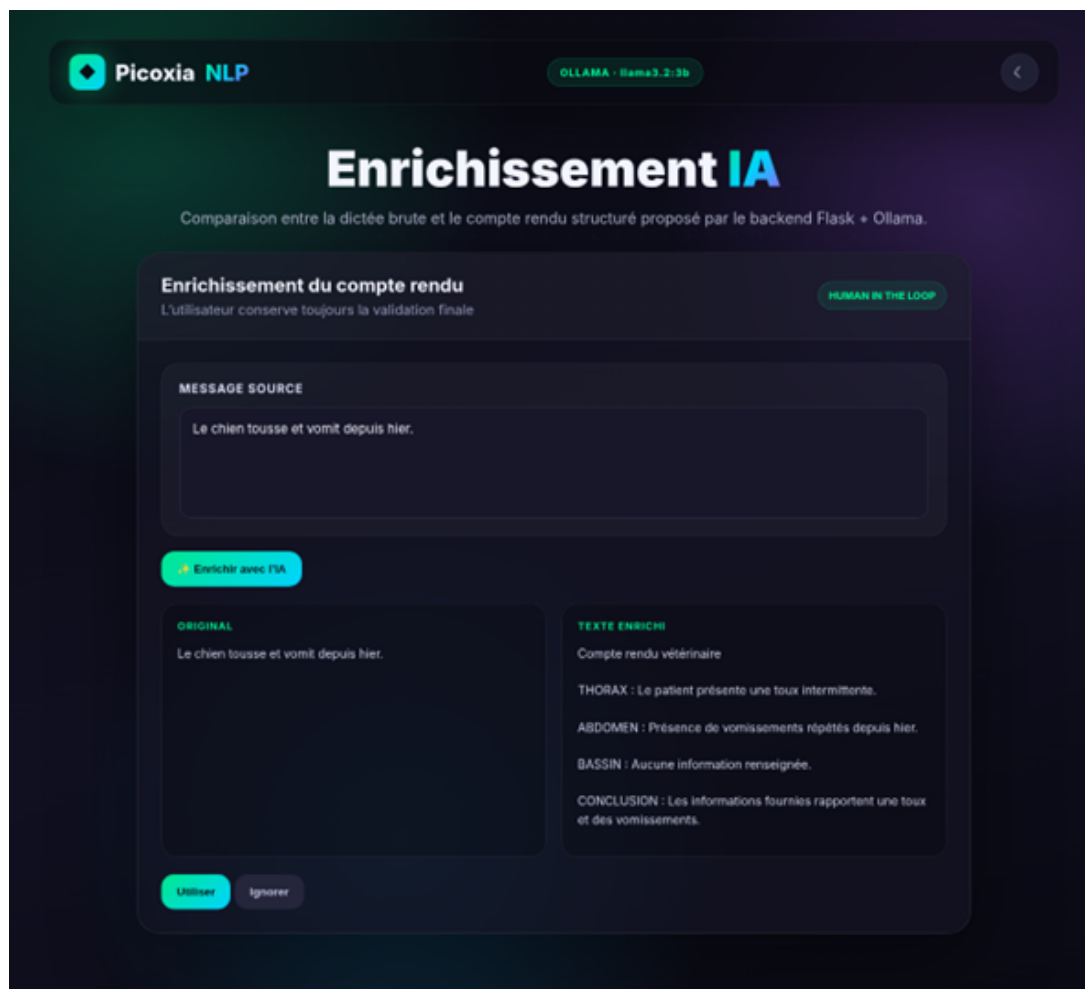


FIGURE 6.1 – Panneau de comparaison avant/après enrichissement

6.4 Gestion du multilingue

La gestion des 14 langues repose sur un fichier source `fr.ts` qui définit toutes les clés de traduction de l'interface. Le script `frontend/scripts/translate.mjs` traduit automatiquement les clés manquantes vers les 13 autres langues via l'API gratuite MyMemory, à chaque `npm run dev` ou `npm run build`. Pour éviter de retraduire à chaque lancement, un hash MD5 de `fr.ts` est mis en cache : si le fichier source n'a pas changé, le script ne fait rien. Les traductions se font en parallèle (pool de 10 requêtes simultanées), et en cas d'échec, la valeur française est utilisée en repli.

Le type `TranslationKey = keyof typeof fr` garantit, à la compilation, que toutes les clés utilisées dans l'application existent bien dans le fichier source. Toutes les nouvelles clés ajoutées pour le module d'enrichissement (boutons, messages d'erreur, etc.) ont été traduites dans les 14 langues via ce mécanisme.

6.5 Intégration MongoDB

L'intégration de MongoDB a été réalisée dans un module séparé `backend/reports.py`, sous la forme d'un blueprint Flask monté sur le préfixe `/api/reports`. Trois endpoints

ont été ajoutés :

- `POST /api/reports` : crée un compte rendu (retourne 201 `Created` avec l'id généré) ;
- `GET /api/reports` : liste les comptes rendus enregistrés ;
- `GET /api/reports/<id>` : récupère un compte rendu précis par son identifiant.

La connexion à MongoDB se fait via `pymongo`, avec les paramètres `MONGODB_URI`, `MONGODB_DB` et `MONGODB_COLLECTION` définis par variables d'environnement (valeurs par défaut : `mongodb://localhost:27017, projet_miage, reports`). Si MongoDB n'est pas joignable, les endpoints renvoient une erreur 500 contenant le message d'exception, ce qui a facilité le débogage pendant le développement.

À ce stade, ces endpoints servent uniquement à persister et récupérer les comptes rendus : ils ne déclenchent pas eux-mêmes la transcription ou l'enrichissement, qui restent gérés par les endpoints `/api/transcribe` et `/api/enrich` décrits précédemment.

7 Difficultés Rencontrées

Cette partie présente les principales difficultés rencontrées pendant le développement de Picoxia, leur impact sur le projet, et les solutions retenues.

Difficulté – Choix du moteur Speech-to-Text

Problème : aucun moteur de reconnaissance vocale ne répondait seul à tous les besoins du projet. Whisper offre la meilleure précision mais nécessite un backend lancé et ne donne pas de retour en temps réel. Vosk fonctionne hors ligne mais avec une précision moyenne sur certaines langues. La Web Speech API donne un retour immédiat mais dépend de Google et ne fonctionne que sur Chrome/Edge.

Impact : se limiter à un seul moteur revenait soit à sacrifier la précision, soit à sacrifier le fonctionnement hors ligne, soit à dépendre d'un service externe pour des données potentiellement sensibles.

Solution : mise en place d'une architecture hybride avec trois moteurs et quatre modes (Auto, Haute précision, Offline, Web Speech), comme détaillé au chapitre 3.

Justification : cette approche laisse le choix à l'utilisateur selon le contexte (présence ou non du backend, besoin de confidentialité, navigateur utilisé), plutôt que d'imposer un compromis unique qui ne convient à aucun cas d'usage.

Difficulté – Gestion des 14 langues

Problème : traduire manuellement l'ensemble de l'interface dans 14 langues (français, anglais, allemand, serbe, grec, néerlandais, russe, ukrainien, italien, finnois, portugais, slovaque, espagnol, tchèque) représentait un travail répétitif et source d'erreurs, en particulier à chaque ajout de nouvelle fonctionnalité (par exemple le module d'enrichissement).

Impact : sans automatisation, chaque nouvelle clé de texte ajoutée à l'interface aurait dû être traduite à la main 13 fois, avec un risque d'oubli ou d'incohérence entre les langues.

Solution : mise en place d'un script de traduction automatique (`translate.mjs`) qui s'appuie sur le fichier source `fr.ts` et l'API gratuite MyMemory pour compléter automatiquement les traductions manquantes dans les 13 autres langues, avec mise en cache pour éviter de retraduire à chaque lancement.

Justification : cette automatisation garantit que toute nouvelle clé ajoutée en français est disponible dans les 14 langues sans intervention manuelle, et le typeage TypeScript (`TranslationKey = keyof typeof fr`) empêche d'utiliser une clé qui n'existe pas.

Difficulté – Choix du modèle d'IA pour l'enrichissement

Problème : le premier modèle testé pour l'enrichissement, `11ama3.2:3b`, ne respectait pas correctement les consignes du prompt : il inventait parfois des symptômes absents

du texte dicté (par exemple une toux non mentionnée par l'utilisateur) et mélangeait les sections du rapport (Anamnèse, Examen, Diagnostic, Traitement).

Impact : pour un document médical, ce type d'erreur est inacceptable, même si le modèle est plus léger et plus rapide. Un rapport vétérinaire enrichi ne doit jamais contenir d'informations inventées.

Solution : le modèle a été remplacé par `qwen2.5:3b`, plus léger (environ 1,9 Go) mais nettement plus fiable que `llama3.2:3b`. Il suit correctement le routage des sections, omet les parties vides plutôt que d'inventer du contenu, et ne produit pas d'hallucinations dans les tests effectués.

Justification : le modèle `qwen2.5:7b`, bien que plus précis, a été écarté car trop lourd et trop lent pour un usage local. Le compromis fiabilité / rapidité offert par `qwen2.5:3b` a été jugé optimal pour un bouton « Enrichir avec l'IA » utilisable en temps réel.

Difficulté – Confidentialité des données médicales

Problème : Picoxia traite des comptes rendus vétérinaires pouvant contenir des informations sur des animaux et leurs propriétaires. Certaines solutions de reconnaissance vocale ou d'IA (Web Speech API, API cloud comme GPT-4o ou Mistral) envoient les données à des serveurs externes.

Impact : utiliser uniquement des solutions cloud aurait posé un problème de conformité avec le RGPD et de confiance pour un usage dans un cabinet vétérinaire.

Solution : les briques principales du pipeline (Vosk pour la dictée hors ligne, Whisper pour la transcription serveur, Ollama pour l'enrichissement) fonctionnent entièrement en local. La Web Speech API, qui envoie l'audio à Google, a été conservée comme mode optionnel et non comme mode par défaut.

Justification : cette approche permet de proposer une option 100 % locale par défaut, tout en laissant la possibilité d'utiliser un mode plus simple (Web Speech) pour des tests ou des cas où la confidentialité est moins critique.

Difficulté – Disponibilité des services locaux (backend et Ollama)

Problème : les fonctionnalités de transcription Whisper et d'enrichissement Ollama dépendent de services lancés localement (serveur Flask sur le port 5000, serveur Ollama sur le port 11434). Si l'un de ces services n'est pas démarré, les appels correspondants échouent.

Impact : sans gestion explicite de ces cas, l'utilisateur se retrouvait face à une interface qui semblait bloquée ou qui échouait silencieusement, sans comprendre pourquoi.

Solution : des endpoints de vérification ont été ajoutés (`GET /api/health` pour Whisper, `GET /api/enrich/health` pour Ollama). Le mode « Auto » de la dictée bascule automatiquement vers Vosk si Whisper n'est pas joignable, avec un message explicite. Pour l'enrichissement, un message d'erreur clair s'affiche si Ollama n'est pas lancé.

Justification : informer clairement l'utilisateur de l'état des services locaux, plutôt que de le laisser face à un échec silencieux, rend l'application plus simple à utiliser et à déboguer pendant les démonstrations.

8 Bilan et Perspectives

8.1 Bilan du projet

Le projet Picoxia avait pour objectif de développer une application web capable de réaliser de la dictée vocale multilingue et de l'enrichissement de texte à l'aide de l'intelligence artificielle. Au terme du projet, une solution fonctionnelle a été mise en place en s'appuyant sur une architecture composée d'un frontend React/TypeScript, d'un backend Flask, d'une base de données MongoDB et de plusieurs technologies de traitement vocal et linguistique.

Les fonctionnalités principales prévues dans le cahier des charges ont été réalisées. L'application permet de sélectionner parmi quatorze langues, d'effectuer une transcription vocale à l'aide de différents moteurs de reconnaissance, d'enrichir automatiquement le texte grâce à un modèle d'intelligence artificielle local et de sauvegarder les résultats dans une base de données. L'intégration de plusieurs moteurs de transcription constitue également un point fort du projet, car elle offre davantage de flexibilité selon le contexte d'utilisation.

Au-delà des aspects fonctionnels, ce projet a permis de mettre en œuvre une architecture complète intégrant des technologies frontend, backend, intelligence artificielle et gestion de données. Les résultats obtenus démontrent la faisabilité d'une solution open source de dictée et d'assistance à la rédaction dans un environnement multilingue.

8.2 Compétences acquises

La réalisation de ce projet a permis de développer plusieurs compétences techniques et méthodologiques. Sur le plan du développement web, elle a renforcé la maîtrise de React, TypeScript, Flask et MongoDB ainsi que la conception d'architectures client-serveur basées sur des API REST.

Le projet a également permis de découvrir des technologies liées à l'intelligence artificielle, notamment la reconnaissance vocale avec Whisper et Vosk ainsi que l'enrichissement de texte à l'aide de modèles exécutés localement via Ollama. L'intégration de ces outils a apporté une meilleure compréhension des contraintes liées aux performances, au traitement multilingue et à la qualité des résultats générés.

Enfin, ce travail a contribué au développement de compétences en analyse des besoins, résolution de problèmes techniques, gestion des dépendances logicielles, tests fonctionnels et documentation d'un projet informatique.

8.3 Limites actuelles

Malgré les résultats obtenus, certaines limites subsistent. La première concerne le mode de transcription hors ligne basé sur Vosk, dont les modèles linguistiques doivent encore

être intégrés et validés pour garantir un fonctionnement complet dans toutes les langues supportées.

Par ailleurs, certaines informations saisies dans l'interface utilisateur ne sont pas encore entièrement exploitées lors de la sauvegarde. Le projet ne dispose pas non plus d'un système d'authentification ni d'une suite de tests automatisés permettant de sécuriser davantage les futures évolutions.

Enfin, les performances de certaines fonctionnalités dépendent fortement des ressources matérielles disponibles, notamment pour les traitements réalisés par Whisper et les modèles d'intelligence artificielle exécutés localement.

8.4 Améliorations futures

Plusieurs pistes d'amélioration peuvent être envisagées afin d'augmenter la robustesse et les fonctionnalités de l'application. Une première évolution consisterait à finaliser l'intégration complète du mode hors ligne et à renforcer la gestion des différents moteurs de transcription.

L'ajout d'une authentification, d'un historique des rapports sauvegardés et de tableaux de bord de suivi permettrait également d'améliorer l'expérience utilisateur. Du côté de l'intelligence artificielle, il serait intéressant d'évaluer différents modèles afin d'améliorer la qualité des enrichissements générés et de réduire les temps de traitement.

Enfin, la mise en place de tests automatisés et d'un processus de déploiement plus structuré contribuerait à renforcer la fiabilité de l'application et faciliterait son évolution vers un environnement de production.

Conclusion Générale

Picoxia est un projet solide, ambitieux et déjà très présentable. Il couvre la majorité du cahier des charges et propose même une approche technique plus riche que les pistes initiales, avec trois moteurs de transcription vocale et un enrichissement IA local via Ollama.

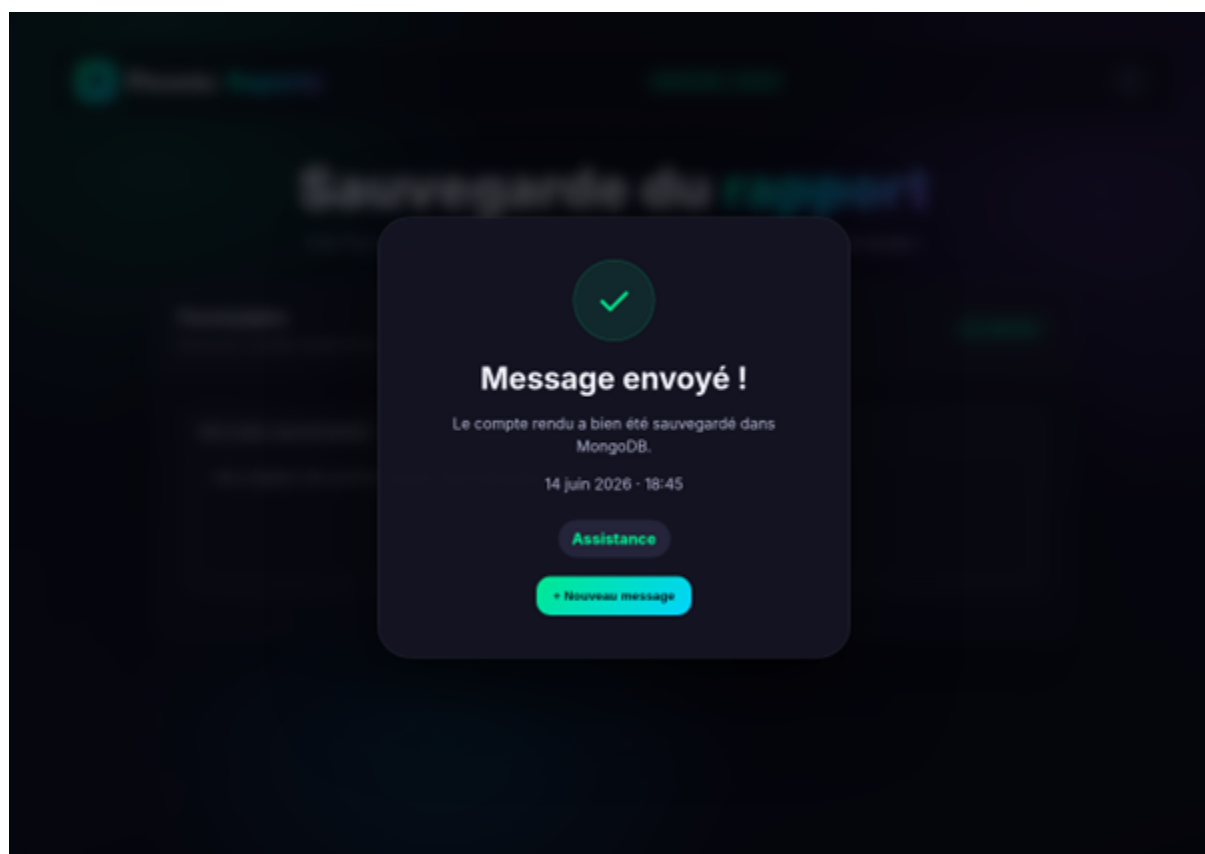
Les principaux atouts du projet sont son architecture claire, son interface très travaillée, son support multilingue homogène, son backend Flask lisible et l'intégration d'une chaîne complète allant de la dictée à la sauvegarde. L'application donne une image moderne et crédible d'un outil d'aide à la rédaction de comptes rendus vétérinaires.

Les faiblesses restantes sont principalement des finitions de livraison et de robustesse : modèles Vosk absents, sauvegarde incomplète du formulaire, configuration de développement, absence de tests et documentation à harmoniser. Ce sont des problèmes corrigeables sans refonte majeure.

En l'état, le projet peut être considéré comme une très bonne preuve de concept. Avec les corrections recommandées, il peut devenir une démonstration finale convaincante, propre techniquement et alignée avec les attentes d'un projet tuteuré MIAGE.

A Captures d'écran

The screenshot displays the Picoxia Voice web application interface. At the top, the logo 'Picoxia Voice' is visible alongside a progress bar with three steps: 1. LANGUE, 2. DICTÉE, and 3. IA. A button labeled 'OPEN SOURCE - 14 LANGUES' is positioned below the progress bar. The main heading reads 'Dictée vocale multilingue et enrichissement IA', with a subtitle stating 'Interface web React + Flask pour dicter, corriger, enrichir et sauvegarder un compte rendu vétérinaire.' Below this, a 'Contactez-nous' section includes a description 'Formulaire intelligent avec dictée et sauvegarde MongoDB' and an 'OPEN SOURCE' button. The form is divided into several sections: 'TYPE ET CATÉGORIE' with fields for 'Type de demande' (set to 'Assistance') and 'Catégorie' (set to 'Medical'); 'LANGUE' with a dropdown set to 'Français (Français)'; 'SUJET' with a text input containing 'Labrador, 5 ans'; and 'DICTÉE VOCALE' which features tabs for 'Auto', 'Whisper', 'Offline', and 'Web Speech', a waveform visualization, and a 'Commencer la dictée' button. A 'MESSAGE DICTÉ' section shows a sample text: 'Patient présenté pour toux intermittente et vomissements depuis hier. Surveillance recommandée.' Below this is an 'Envoyer' button. At the bottom, a row of flags represents the supported languages.



B Guide d'installation

Ce chapitre présente les étapes nécessaires pour installer et lancer l'application *Picoria*. Le projet est composé d'un frontend React et d'un backend Python Flask, avec des services externes pour la transcription vocale, l'enrichissement NLP et la base de données.

B.1 Prérequis

Avant de commencer, il faut installer les éléments suivants :

- Node.js et npm pour le frontend.
- Python 3.10 ou supérieur pour le backend.
- MongoDB, en local ou accessible à distance.
- Ollama pour le module d'enrichissement NLP.
- Un navigateur web moderne.

B.2 Installation du frontend

1. Ouvrir un terminal dans le dossier `frontend`.
2. Installer les dépendances :

```
npm install
```

3. Lancer l'application en mode développement :

```
npm run dev
```

4. Ouvrir l'adresse fournie par Vite dans le navigateur.

B.3 Installation du backend

1. Ouvrir un terminal dans le dossier `backend`.
2. Créer un environnement virtuel Python :

```
python -m venv .venv
```

3. Activer l'environnement virtuel sous Windows :

```
.venv\Scripts\activate
```

4. Installer les dependances :

```
pip install -r requirements.txt
```

5. Lancer le serveur Flask :

```
python app.py
```

B.4 Configuration des services externes

- **MongoDB** : le backend utilise la variable `MONGODB_URI` pour se connecter a la base de donnees.
- **Ollama** : installer Ollama, puis demarrer le service avec `ollama serve`.
- **Modele Ollama** : telecharger un modele compatible, par exemple `ollama pull llama3.2:3b` ou `ollama pull mistral`.
- **Whisper** : la transcription vocale repose sur `faster-whisper` cote serveur.

B.5 Verification

Une fois l'installation terminee :

- le frontend doit etre accessible dans le navigateur,
- le backend doit repondre sur le port 5000,
- les endpoints d'API doivent etre disponibles pour la transcription, l'enrichissement et la sauvegarde des comptes rendus.

C Guide utilisateur

Cette section explique comment utiliser l'application *Picoxia* pour saisir un message, dicter du texte, enrichir un compte rendu et sauvegarder le resultat.

C.1 Acces a l'application

1. Demarrer le backend Flask.
2. Demarrer le frontend React.
3. Ouvrir l'interface web dans le navigateur.

C.2 Remplissage du formulaire

L'utilisateur complete les champs principaux suivants :

- le type de demande,
- la categorie,
- la langue,
- le sujet,
- le message ou compte rendu.

C.3 Utilisation de la dictée vocale

L'application propose trois modes de transcription :

- **Auto** : le systeme tente d'utiliser Whisper, puis bascule vers Vosk si le service n'est pas disponible.
- **Whisper** : transcription cote serveur pour une meilleure precision.
- **Offline** : transcription locale avec Vosk.
- **Web Speech** : transcription via l'API de reconnaissance vocale du navigateur.

Pour dicter un texte :

1. Choisir le mode de dictée.
2. Cliquer sur le bouton de demarrage.
3. Dicter le contenu a voix haute.
4. Arreter l'enregistrement pour valider le texte transcrit.

C.4 Enrichissement du texte

Une fois le texte saisi, l'utilisateur peut lancer l'enrichissement NLP.

1. Cliquer sur le bouton d'enrichissement.
2. Attendre la generation du bilan structure.
3. Comparer le texte original et le texte enrichi.
4. Choisir de conserver ou non la version enrichie.

C.5 Sauvegarde du compte rendu

Quand le formulaire est complet, il suffit de cliquer sur *Envoyer*. Le backend enregistre alors le compte rendu dans MongoDB avec :

- le texte original,
- le texte enrichi,
- la langue,
- le type de demande,
- la date de creation.

C.6 Resultat

Après l'envoi, une fenetre de confirmation affiche le succes de l'enregistrement et permet de recommencer un nouveau message.